Reflexión personal – Ian Armando Borde Escobar A00846007

En esta actividad relacioné las estructuras de datos con tareas concretas de una aplicación, partí de estructuras que ya tenía, las adapté, preparé pruebas e integré el programa, la IA me ayudó con las validaciones, las búsquedas y las estadísticas, también vi que puede agregar funciones innecesarias, como las comprobaciones adicionales de orden y estabilidad, al comparar la propuesta con las instrucciones pedí retirar esos extras y conservé las verificaciones durante las pruebas y sin la ayuda de la IA habría dedicado más tiempo a consultar ejemplos y probar cada función y hacer validaciones

La lista ligada simple es la base de la Pila y la Fila, en la Pila se inserta y elimina al inicio para obtener un comportamiento LIFO, en la Fila se inserta al final y se elimina al inicio para obtener FIFO, head y tail permiten realizar estas operaciones en O(1), la Fila procesa las líneas en orden de llegada, después de desencolar cada una se valida y convierte en un registro, las líneas incorrectas se reportan y las válidas se guardan al final de la lista doblemente ligada

Elegí Merge Sort porque permite ordenar la lista mediante sus enlaces, sin copiar los registros a un arreglo, su comportamiento es O(n log n) y conserva el orden de llegada en los empates al elegir primero el nodo izquierdo durante la mezcla

Para la opción 3 utilicé una Pila de apuntadores a las coincidencias, push y pop permiten verlas en orden inverso, aunque también podía localizar el último registro del día y regresar usando prev, la Pila cumple el requisito y hace explícito el comportamiento LIFO, pero necesita memoria adicional O(k), recorrer con prev evita esa memoria, aunque requiere localizar el último registro y detenerse cuando cambie la fecha

La búsqueda binaria necesita consultar el elemento intermedio, en una lista ligada llegar a él requiere recorrer enlaces, por eso no se obtiene directamente el tiempo O(log n) de un arreglo, para hacerlo posible podría construir un índice de apuntadores, con un costo de O(n) en tiempo y memoria, después buscaría los límites del día en O(log n), pero tendría que mantener ese índice si cambia la lista

Ambos archivos tenían 6818 registros, el primero necesitó 78289 comparaciones y el segundo 49551, el segundo también terminó más rápido, esto muestra que estar casi ordenado no significa menos comparaciones en esta implementación, ambos alcanzaron una profundidad de 14 niveles de llamadas recursivas, cada llamada ocupa espacio en la pila de llamadas, distinta de mi Pila de la opción 3, la profundidad crece en O(log n) porque Merge Sort divide la lista aproximadamente a la mitad